Network Routing Optimization Report

Eli DeSha, Sean McKnight, Erick Ramirez

Github Repo: https://github.com/seanmcknight182/NetworkRoutingOptimization/tree/main

Overview

Network routing optimization (NRO) is the process of finding the shortest path from one place to another. Whether it be over a network, within a server, or even using a GPS, NRO is integral to our daily lives. We displayed several methods of solving these sorts of problems by using a Benchmark algorithm to determine the comparative speed of several other algorithms. This allowed our results to be concise and informative.

Objective/Research Questions

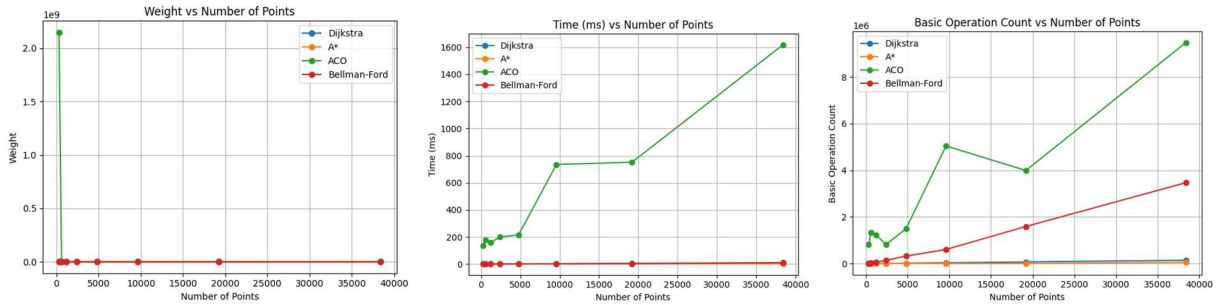
- Why is Dijkstra's algorithm typically used in networks instead of heuristic algorithms?
- How much faster are heuristic algorithms than exact ones, and is the time saved worth the efficiency cost?
- How can we find the shortest distance from one point to another?

Experiment Description

The experiment was conducted across eight node sizes: 300, 600, 1200, 2400, 4800, 9600, 19200, and 38400 nodes. For each node size, the data generator produced a new randomized graph saved to a shared data file. This file was then passed to each of the four algorithms: Dijkstra's, A*, ACO, and Bellman-Ford. Every algorithm reads the same graph data and computed the shortest path from the first node to the last node. Each algorithm recorded three metrics upon completion: the time taken in milliseconds, the number of basic operations performed, and the total weight of the shortest path found. These results were saved to individual output files and later aggregated by a Python script which produced comparative graphs across all four algorithms for each metric.

The three exact algorithms (Dijkstra's, A*, and Bellman-Ford) were each run once per node size since they are deterministic and will always produce the same result. ACO, being a metaheuristic, was also run once per node size, though its results may vary between runs due to it being nature-based.

Results & Analysis



All algorithms were tested across 8 node sizes from 300 to 38,400 nodes

- **Weight:** Dijkstra's, A*, and Bellman-Ford all found the shortest path at every size, with their weights being the same and near zero on the scale of the graph, ACO was a massive outlier at 300 nodes, producing a weight of approximately 2.15 billion before dropping down, meaning it failed to find a valid path at the smallest size and struggled to find the right path
- **Time:** A* and Dijkstra's were basically the same on the graph, both staying near 0ms even at 38,400 nodes, Bellman-Ford also stayed low, never passing 50ms. ACO was the extreme outlier, starting at around 175ms at 300 nodes, spiking to 750ms at 9,600 nodes, and reaching about 1,620ms at 38,400 nodes
- **Operations:** Dijkstra's and A* were nearly flat across all sizes, staying close to zero relative to the scale. Bellman-Ford grew linearly reaching 3.5 million operations at 38,400 nodes, while again ACO was the worst performer, peaking near 5 million at 9,600 nodes, dipping, then surging to 9.3 million at 38,400 nodes
- **Overall:** A* and Dijkstra's dominated all three metrics — ACO was the weakest performer in every category, with its weight anomaly at 300 nodes being a significant red flag worth noting

Sources

Some of the sources we used were ChatGPT to fix issues with the code of A* and Bellman-Ford algorithms, as well as helping us create both the Makefile and Script.py. For the creation of the ACO (Ant Colony Optimization) algorithm the help of Geekforgeeks, multiple videos, and part of Baeldung code was used (but this code was in Java, so it was mainly used as a guideline). When we got stuck on a problem, we tried to use the ai to see if it could detect what the issue was and maybe spot an easy solution. It worked for simple problems like the script not placing the results in the correct places. But more in-depth issues had to be fixed manually.

https://www.geeksforgeeks.org/machine-learning/introduction-to-ant-colony-optimization/
https://www.youtube.com/watch?v=1qpvpOHGRqA
https://www.youtube.com/watch?v=783ZtAF4j5g
https://www.youtube.com/watch?v=u7bQomllcJw&t=16s
https://www.youtube.com/watch?v=qfeymoF8pb4&t=1s
https://www.baeldung.com/java-ant-colony-optimization

Workload table per teammate

Labels	**Sean**	**Erick**	**Eli**
Coding	45%	35%	20%
Coordination	20%	20%	60%
Clerical work	30%	30%	40%
Overall	33.3%	33.3%	33.3%

Difficulties

The only major problem we faced during this project was the lack of concrete output file format. This caused us to need to rewrite our code many times to accommodate for the new specifications that would work better for certain circumstances.